Software Engineering Task-2 Report

Course Name: Introduction to Software Engineering

Course Code: IS1103-1

Assignment Title: System Modeling and Agile Methods for Ride-Hailing System

Student Name: B. Sunidhi Udupa

USN: NNM24IS054

Semester & Section: 4 A

Department: Information Science & Engineering

Institution: NMAM Institute of Technology

Instructor: Dr. Jason Elroy Martis

Academic Year: 2025–2026

Date: 05-04-2026

Abstract

The design of a ride-hailing application is the subject of this report. Through a platform, the ride-hailing system links drivers and passengers. Ride booking, tracking, and payment processing are all handled by this platform. We employed system modeling methods such as state diagrams, sequence diagrams, and context models. The behavior and structure of the system are depicted in these diagrams. The boundaries of the system are displayed in the context model. Step-by-step communication between system components is explained by the interaction model. The ride-hailing system's states throughout a ride's lifecycle are depicted in the behavioral model. Scrum was also utilized to organize the development process. Clarity in requirements and quality control are ensured by user stories and the definition of done. This paper demonstrates how Agile methodologies and system modeling aid in the creation of software systems that are maintainable, such as the ride-hailing system.

Github repository link

https://github.com/Sunidhi70/Task-2/tree/main

Introduction

Software engineering offers techniques for creating and designing software systems. For ride-hailing services to comprehend interactions and workflows, appropriate system modeling is required. The design of a ride-hailing system using UML models and implementation techniques is the main goal of this assignment. The goal of the ride-hailing platform is to link drivers and passengers while guaranteeing payment, tracking, and booking features.

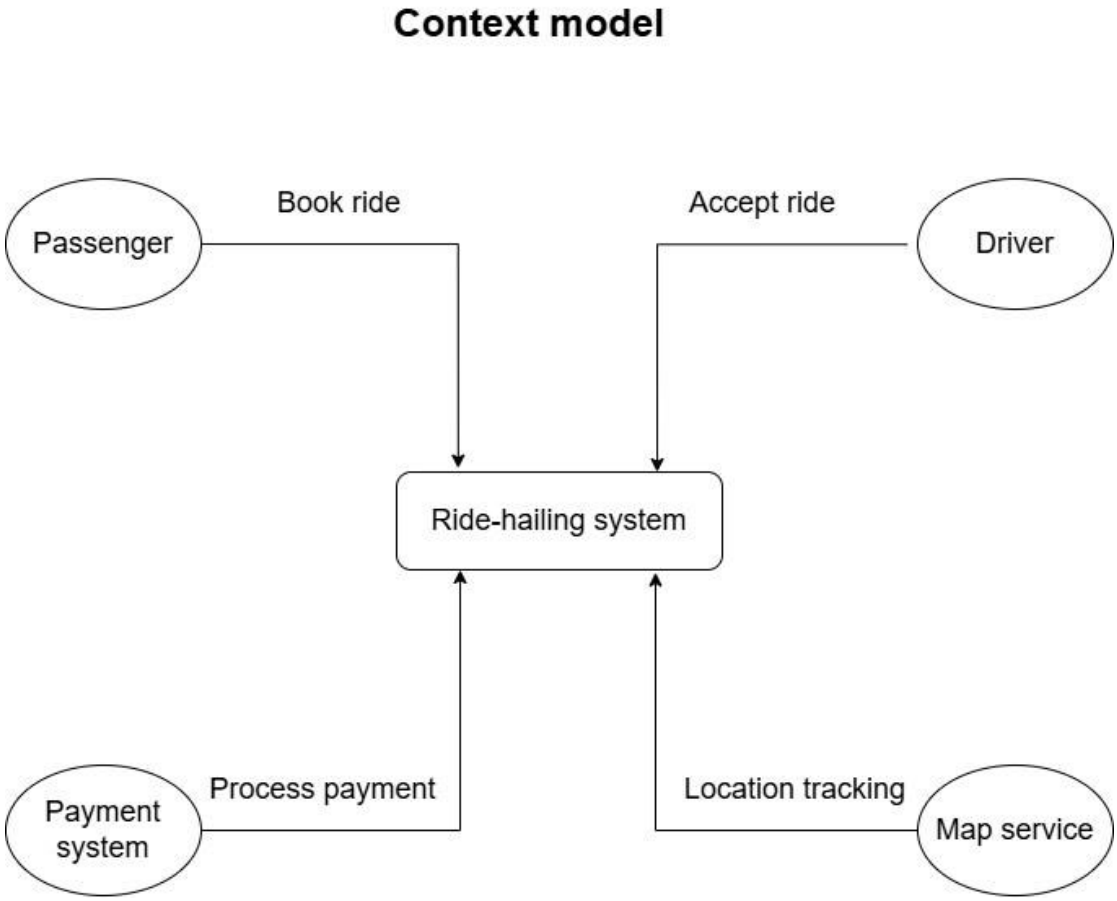
Problem statement

The objective is to create a ride-hailing platform similar to Uber. Passengers must be able to reserve rides through the ride-hailing system, drivers must be able to accept requests, and the system must handle payments, tracking, and communication. The anticipated result is a system design backed by Agile development planning and UML diagrams.

Modeling of Systems

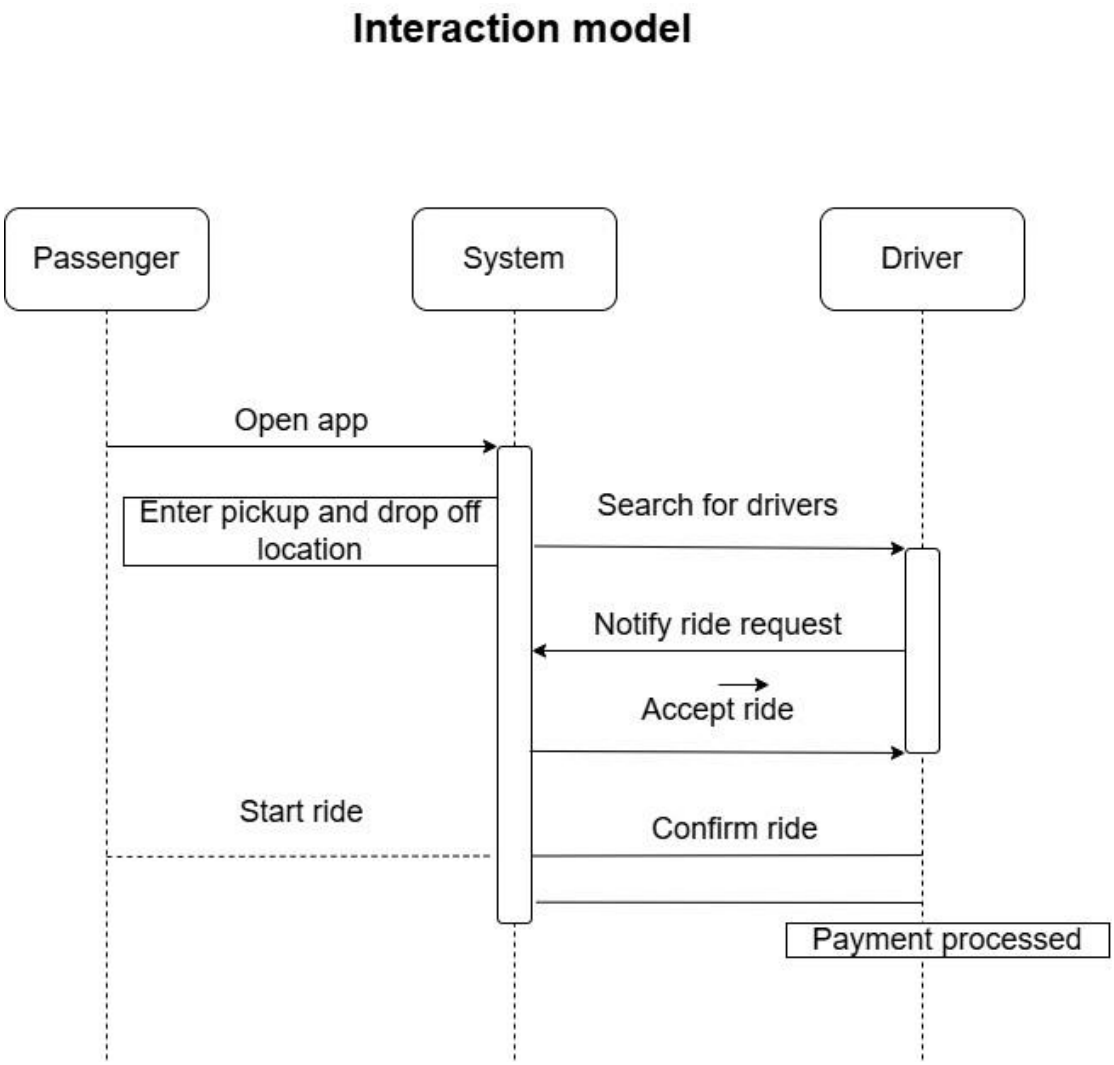
The Context Model

The ride-hailing system is depicted in the context model as an entity that interacts with elements like the GPS service, payment system, driver, and passenger. It draws attention to how these entities exchange data. Rides are requested by the traveler. Confirmations are received. After receiving ride details, the driver modifies the ride status. the GPS service and payment system. navigating.



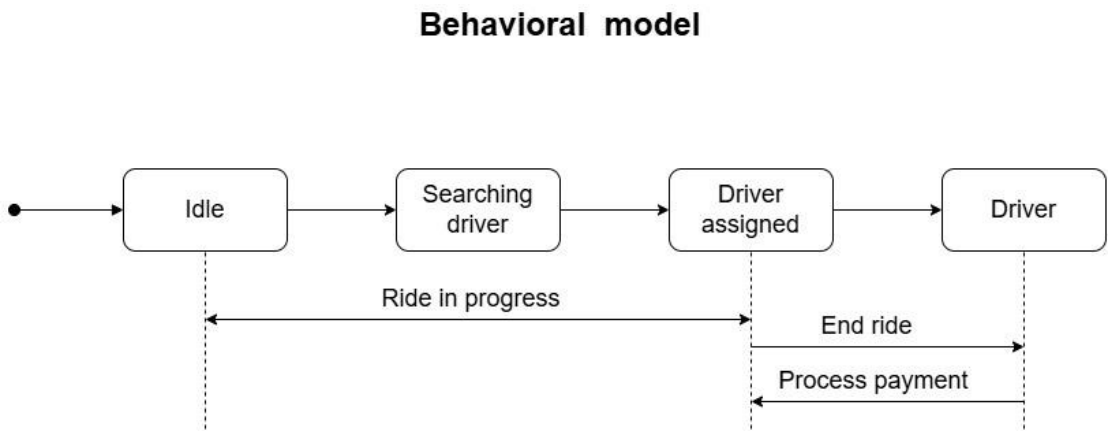
Sequence Diagram: Interaction Model

The sequence diagram illustrates how the ride booking process works. The process begins with the passenger making a ride request, which is followed by system processing, driver matching, ride confirmation, ride execution, and payment completion. Real-time communication between system components is visualized with the aid of this model.



Model of Behavior (State Diagram)

States like Idle, Searching Driver, Driver Assigned, Ride Started, Ride Completed, and Payment Done are depicted in the state diagram. Events like booking request driver acceptance, ride completion, and successful payment are the basis for transitions. This model aids in comprehending the behavior of the ride-hailing system.



Agile Methods

Overview of Scrum

Development is divided into sprints using the Scrum methodology. It encourages cooperation, adaptability, and ongoing development.

Planning for Sprints

User registration, login features, and basic user interface design are all included in Sprint 1. Ride booking, driver matching, and payment integration are the main topics of Sprint 2. The ride-hailing system's components are delivered during each sprint.

User Narratives

Passenger Stories: In order to utilize the ride-hailing system, I would like to register as a passenger. I want to reserve a ride as a passenger so that I can go. I want to track

my ride as a passenger so that I can find out where I am. Driver Stories: In order to make money as a driver, I would like to take rides. As a driver, I need navigation assistance to get to my destination quickly.

Definition of done

When the code is implemented, tested, error-free, user-friendly, and approved, a feature is deemed complete.

User Narratives with a Done Definition

User Stories for Passengers

User Enrollment:

User Story: I would like to create an account as a passenger in order to utilize the ride-hailing service.

Definition of Done: The passenger can enter information, data is saved, and there are no errors or duplicate confirmations displayed.

Functionality for Login:

User Story: I want to access my account as a Passenger by logging in.

Definition of Done: If the login is successful, the session is maintained and redirected; if not, an error is displayed.

Get a Ride:

User Story: I want to reserve a ride so that I can travel as a passenger.

Definition of Done: Drivers are sent, locations are entered, and requests are validated. Verification is displayed.

Track Ride:

User Story: I want to track my ride as a passenger so I can find out where the driver is.

Definition of Done: The route is correct, the map is updated in real-time, and driver information is displayed.

Pay:

User Story: I want to pay as a passenger once the ride is over.

Definition of Done: The fare is computed, the payment is processed, and a receipt is provided. Failure is managed.

User Stories for Drivers

Driver Enrollment:

User Story: I would like to register as a driver in order to offer ride services.

Definition of Done: Information is input, an account is created, verified, and a confirmation is displayed.

Accept or Deny a Ride:

User Story: I want to be able to accept or reject ride requests as a driver.

Definition of Done: The passenger is informed, work is accepted or rejected, and status updates are received.

Help with Navigation:

User Story: I want assistance with navigation as a driver.

Definition of Done: Alternative routes are provided; the route is displayed using GPS. The time and distance are shown.

Ride Status Update:

User Story: I would like to update the ride status as a driver.

Definition of Done: Updates that have been started or finished are visible. Errors do not exist.

View Your Earnings:

User Story: As a driver, I'd like to see my earnings.

Definition of Done: Earnings are displayed as updated. No mistakes are present. It is stored securely.

Discussion

The ride-hailing system's architecture and behavior were made easier to see with the aid of system modeling techniques. UML diagrams were used to guarantee developer comprehension and communication. Agile methods made it possible for ongoing feedback and iterative development. Diagram design was one of the difficulties. guaranteeing uniformity among models.

Conclusion

This project shows how to use Agile methodology and system modeling to design a ride-hailing system. While Agile guarantees development, the models offer insight into the lifecycle and interactions of the ride-hailing system. A scalable system design similar to the ride-hailing system is produced by combining these strategies.

References

[1] R. S. Pressman and B. R. Maxim, Software Engineering , A Practitioner's Approach, 8th ed., New York, USA: McGraw-Hill, 2015.

[2] I. Sommerville, Software Engineering, 10th edition, Boston, MA, USA: Pearson, 2016.

Declaration

I hereby declare that this assignment is my original work and has not been copied or plagiarized from any source. All references and resources used have been properly cited. Any ambiguities will lead to forfeiting my marks.

Student Signature:

Date: 05-04-2026